Speaker notes

Q&A-prep reference for the presentation. All numeric stats verified against `final_report/methodology/methodology.tex` and `final_report/results/results.tex`.

DEMO

START THE TOOL FIRST validateCustomer() calculateTaxedAmount() calculateFinalTotal()

1. Show the score + what could have been
2. Show how the dots are different code states over time. Red build / tests failed.
3. Have the different stats can toggle between -> while code cleanliness increases strictly, process score does not.
4. Show the flashing divergence point on the extract method -> can see that the tool has detected this from the code diffs.
5. Click the “IDE replay would have scored” indicator to highlight the alt path + see the less steps + less LOC of churn.

Part 1 — Score formula

Reference for the **Scoring a trajectory** slide (visible) and the **Cleanliness sub-score** slide (hidden).

Table 1 — J(τ) terms

Term	Weight	What it measures	Why this term & weight	Stats support
β (baseline)	**50**	Anchor of the `[0, 100]` clamp. A trajectory with no cleanliness gain and no penalties scores exactly 50.	Symmetric baseline — clean trajectories rise toward 100, penalised ones fall toward 0. Lets trajectories of different lengths sit on one scale. *(methodology.tex:64–71)*	Not a fitted weight. Monotone-recovery confirms sum/sum $\to$ 0 when all terms stripped, so the score’s signal comes from the weighted terms, not the baseline. *(Table `tab:results-ablation-monotone`)*
`W_g · ΔC(τ)` — cleanliness gain	**+50**	Endpoint cleanliness change: `ΔC = C(s_T) − C(s_0)`. Each unit of gain adds 50 points.	Main positive term. Cedrim et al. show refactoring often leaves structural metrics unchanged or worse, so improvement must be rewarded explicitly. Paixão et al. justify treating cleanliness gain and process penalties as competing parts of one score. Sized high so a plausible single-step gain can’t outweigh a broken build. *(methodology.tex:111–117; cedrim2017refactoring; paixao2017balancing)*	• Solo recovery = **0.000** on all sets (cancels — user and alternative validate against same terminal AST).• LOO recovery >1.0 (1.060 inj, 1.120 us, 1.108 agent) — it matters via clamp interaction.• Single-knob top-1 disruption = **10.7%** — largest of all weights. *(Tables `tab:results-ablation-solo`, `tab:results-ablation-loo`, `tab:results-sensitivity-knobs`)*
`W_l · σ_l(τ)` — step-savings bonus	**+11**	Step-savings bonus credited only to alternatives that reach the same end state in fewer steps. The user trajectory is never penalised for length.	Mirrors search-based refactoring’s multi-objective treatment of operation count. Rewards efficient alternatives without punishing necessarily-long user paths. Weight equal to W_mi and W_lag, half of W_b — literature-severity ordering. *(methodology.tex:127–128; harman2007pareto; mohan2019manyobjective)*	• Solo recovery = **0.426** on user-study — **largest single-term contributor**.• LOO drops user-study magnitude to 0.636 — biggest leave-one-out.• LOO Kendall τ = **0.527** on user-study — the clearest rank-carrier.• Single-knob top-1 disruption = 6.2%. *(Tables `tab:results-ablation-solo`, `tab:results-ablation-loo`, `tab:results-ablation-user-loo`)*

Term	Weight	What it measures	Why this term & weight	Stats support
`W_b · r_b(τ)` — broken build / test rate	**−28**	Fraction of *wall-clock* time the build or tests were broken: `r_b = b_ms / e_ms`. Time-weighted so it's insensitive to sample rate.	Broken state = behaviour preservation temporarily abandoned — the largest single penalty weight. Paixão et al. show developers give up ~25% of available metric improvement to avoid disruptive intermediates. Counter-evidence acknowledged honestly: Gautam et al.'s RefactorBench shows rejecting every broken intermediate can hurt agent performance on multi-file edits. *(methodology.tex:114–117; paixao2017balancing; gautam2025refactorbench)*	• Solo recovery = **0.302** (inj), 0.225 (us) — 2nd-largest on injection set.• LOO Kendall τ = **0.192** on injection — strong rank-carrier in controlled setup.• Single-knob top-1 disruption = 5.3%. *(Tables `tab:results-ablation-solo`, `tab:results-ablation-loo`)*
`W_st · r_st(τ)` — skipped-test rate	**−14**	Laplace-smoothed event-rate of 60-second refactoring windows with no test run: `r_st = (k+1)/(n+2)`.	Skipping tests removes the developer's main feedback signal that behaviour really has been preserved. 60-s window motivated by Murphy-Hill et al.'s finding that developers test in batches, not after every edit. Weight = ½ × W_b: weaker evidence of process damage than an observed broken state. *(methodology.tex:119–120; murphyhill2009howrefactor)*	• Solo recovery = **0.371** on user-study — 2nd-largest, *bigger than broken* on natural traces. Report attributes this to "the more natural user traces expose test-running behaviour more clearly than the controlled injection set".• LOO τ = 0.663 on user-study.• Single-knob top-1 disruption = 6.7%. *(Table `tab:results-ablation-solo`)*
`W_mi · r_mi(τ)` — manual edits the IDE could refactor	**−11**	Laplace-smoothed rate of manual edits accomplishing what an IDE refactoring could have done. Detected by RefactoringMiner over sliding commit windows in the shadow repo.	Hand-edits bypass IDE precondition checks, making subtle behavioural slips easier. Negara/Vakilian confirm developers often refactor manually even when an automated equivalent exists. **Weight not fitted** — no published study links fault rate to manual-vs-IDE; follows literature-severity ordering. *(methodology.tex:122–125; mens-tourwe; negara2013; vakilian2012)*	• Solo recovery = 0.261 (inj), 0.170 (us), **1.000 (agent)** — 41/42 agent DPs are Manual-Refactor; treat with care, it's a detector-shape artefact on agents, not a general property.• LOO τ = **0.588** on user-study — 2nd-largest rank effect after length.• Single-knob top-1 disruption = 7.6%. *(Tables `tab:results-ablation-solo`, `tab:results-ablation-user-loo`)*
`W_lag · ℓ(τ)` — intermediate cleanliness lag (degradation)	**−11**	`ℓ(τ) ∈ [0, 1]`: per-checkpoint running mean of `max(0, C(s_T) − C(s_i))`. Positive while the trajectory sits below its final cleanliness — penalises time in degraded intermediate states.	Endpoint gain alone can't tell apart two trajectories with the same endpoints but different intermediate quality. Adds a delay cost — Cunningham's technical-debt metaphor and Kruchten et al.'s formalisation both treat degraded code as more costly the longer it remains. Cedrim et al. show intermediate states can degrade structural metrics even when the final result improves. Weight = W_l = W_mi, half W_b. *(methodology.tex:130–133; cunningham1992wycash; kruchten2012technicaldebt; cedrim2017refactoring)*	• Lowest solo recovery: 0.132 (inj), **0.014** (us).• Single-knob top-1 disruption = **0.4%** (changes top-1 only once).• LOO τ = 0.784 — smallest user-study rank effect.• **But:** positive-magnitude Ordering DPs rise from 4 → 10 of 24 once W_lag is included — its real value is in the Ordering detector. *(methodology.tex:135; Tables `tab:results-ablation-solo`, `tab:results-sensitivity-knobs`)*

Term	Weight	What it measures	Why this term & weight	Stats support
`W_cg · n_cg(τ)` — long durations without commits	**−7**	Count of commit-gap events. One event per stretch of ≥ 6 green-refactor checkpoints with no user commit.	Smallest process weight — a deliberately narrow claim. Captures the review/rollback cost of bundled changes, **not** merge speed (Kudrjavets et al. show no correlation between PR size and time-to-merge across 1.25M PRs). Tao & Kim show separated commits aid review; Di Biase et al. found decomposed changes cut false-positive review comments (1 vs 6, p = 0.03); Herzig et al. show tangled commits hurt defect attribution. **Weight not fitted** — no commit-cadence-to-cost study exists. *(methodology.tex:138–142; taokim2015partitioning; dibiase2019decomposition; herzig2013tangled; kudrjavets2022small)*	• Solo recovery = 0.080 (inj), 0.125 (us).• LOO recovery = 0.943 / 0.900; mean LOO τ = 0.926 — affects ranking modestly while leaving most magnitude intact.• Single-knob top-1 disruption = 6.7%. *(Tables `tab:results-ablation-solo`, `tab:results-ablation-loo`)*

Table 2 — Cleanliness sub-score (6 equally-weighted = 1.0)

Signal	Tool / aggregation	What it measures	Why this signal	Stats support
Cognitive complexity	SonarSource cognitive complexity, per-method mean	Method-level control-flow comprehension intricacy.	Captures readability/comprehension that pure size metrics miss. Muñoz Barón et al. validate it against 24,400 human ratings (positive correlations with comprehension time + subjective ratings). Counterpoint: Lavazza et al. find it predicts understandability "about as well as size + cyclomatic complexity" — kept as one partial signal in the 6-way composite. *(methodology.tex:176–177; campbell2018cognitive; munozbaron2020cognitive; lavazza2023cognitive)*	Group-level: cleanliness sub-weights perturb top-1 ranking in at most **0.9%** of cases — an order of magnitude less than process weights. *(methodology.tex:171; Table `tab:results-sensitivity-knobs`)*
Coupling — CBO	CK library, per-class mean	Number of classes a class depends on.	Classical structural-design signal: lower coupling = better localisation. Foundational metric from Chidamber & Kemerer, widely used as a search-based-refactoring objective. *(chidamberkemerer1994ckmetrics; harman2007pareto)*	2 top-1 changes / 225 perturbation cases = **0.9%** — minimal sensitivity. *(Table `tab:results-sensitivity-knobs`)*
Duplication — CPD	PMD CPD, lines on touched files	Lines of code identified as duplicated.	Repeated code = missing abstraction; high maintenance cost (every copy must be updated together). Treated as a design problem + refactoring motivator. *(heitlager2007sig; fowler1999refactoring)*	No measurable top-1 disruption when its sub-weight is perturbed. Six-part composition "makes the score less dependent on any single metric". *(methodology.tex:147)*

Signal	Tool / aggregation	What it measures	Why this signal	Stats support
Readability	Buse & Weimer feature blend, line-weighted (line length, indentation, identifier characteristics)	Lexical / textual readability — features structural metrics miss.	Combining structural + textual features aligns better with human readability judgements than structural-only models. **Caveat:** uses the *feature taxonomy*, not the trained model — calibrating the trained model would need domain-specific data we haven't collected. *(methodology.tex:176; scalabrino2018readability; buse2010readability)*	Not isolated in the sensitivity table; bundled into the 6-signal composite. Group-level effect ≤ 0.9% top-1.
Code smells — PMD	PMD violation count on touched files	Rule-based heuristic indicators of possible design problems.	Smells are widely used as refactoring motivators. Foundational theory from Marinescu, Arcelli Fontana et al., Moha et al. **Caveat:** smell detectors disagree on precision/recall, so PMD is used as one signal among several rather than as ground truth. *(background.tex:42–44; marinescu2004; arcellifontana2016falsepositives; moha2010decor; fowler1999refactoring)*	Not isolated in the disruption table. Six-metric averaging dilutes single-tool noise — intentional design. *(methodology.tex:147)*
Cohesion — TCC	CK library, per-class mean; dropped for classes with < 2 methods	Tight Class Cohesion — degree to which class methods use shared state.	High cohesion = well-designed class; low cohesion suggests the class should be split. Complements CBO (inter-class) by capturing intra-class integrity. Widely used in search-based refactoring. *(biemankang1995tcc; harman2007pareto)*	Not isolated in disruption table. Group-level effect ≤ 0.9% top-1.

Cross-cutting footnotes (memorise these)

- **Top-1 robustness.** Single-knob sweep: **96.5%** of user-study rankable cases keep their top-1 DP under any 1-weight perturbation. Clamp-frozen rate just 1.8% — so almost all of the stability is genuine perturbation response, not a clamp artefact. Multi-knob sweep ($\sigma = \ln 2$, covers $\approx \times 0.25 – \times 4$ of production weights): top-1 drops to **84.6%**, mean Kendall τ = **0.586**.
- **Cleanliness vs process weights.** Perturbing any single cleanliness sub-weight changes top-1 in **≤ 0.9%** of cases. Process weights change it in 5.3 – 10.7% of cases. ≈ 10× difference — supports the equal-weight cleanliness design.
- **Honest scope limit.** Weights are **not fitted** to outcome data. They follow the literature-severity ordering set in `methodology.tex:80`. Full predictive validation would require an external ground-truth dataset of longitudinal code-quality / maintenance outcomes, which does not currently exist. The score is **defensible locally** — robust to plausible weight perturbations — not claimed to be globally optimal.

Part 2 — Ordering DAG: dependency analysis between refactoring steps (Slide 11)

Effects: four sets per step (`SpecEffects.effects0f`)

Each spec is statically reduced to four sets of *entities* (Type / Method / Field / Package, keyed by FQN):

set	meaning	example
`reads`	entities referenced	RenameClass reads the old type
`writes`	entities modified	RenameMethod writes the declaring type (call-sites)
`produces`	new entities created	ExtractMethod produces a new method
`consumes`	entities destroyed	RenameClass consumes the old type

Four edge rules — for each pair (i, j) with i < j

If *any* of these fires, add edge i → j:

1. **READ-AFTER-WRITE / READ-AFTER-CONSUME** — j reads X, i produced / wrote / consumed X.
2. **WRITE-AFTER-WRITE** — j writes X, i wrote or consumed X.
3. **CONSUME-AFTER-CONSUME** — both i and j consume X.
4. **PRODUCES-AFTER-CONSUME** (cross-range, via `SpecVersioner`) — j re-creates X under a new SSA version after i consumed the previous version.

SSA-style versioning

`SpecVersioner` tracks **live ranges** per entity key. Producing opens a new version; consuming closes the current one. Reads/writes are stamped with the live version. Two reads of "method foo" don't conflate if they refer to *different* live versions — so renaming foo → bar → foo doesn't introduce spurious dependencies.

Why this is "necessary but not sufficient"

The DAG only encodes *known* dependencies from spec metadata. After enumeration, each candidate ordering is replayed on a borrowed git worktree, and the **terminal AST is hashed** against the user's terminal AST (`ReorderSynthesiser.checkTerminalDivergence`). If the hash doesn't match, the ordering is discarded — so a too-permissive DAG (missing a real dependency) is caught at the validation gate, not silently accepted.

Part 3 — Is the process score reliable?

Reference for the **"Is the process score reliable?"** setup slide and the **"Score is locally robust"** results slide.

- **Rankable subset** — sessions with ≥ 2 detected divergence points only (15 injection / 25 user-study / 20 agent). Sessions with 0 or 1 DP have a mechanically τ = 1.0 ranking (no order to perturb), so they're excluded from rank stats to avoid inflating the headline numbers.
- **Magnitude vs ranking** — magnitude tables use the **full** session sets (45 / 30 / 48), since per-DP magnitude is well-defined on every session regardless of DP count. **Rank** statistics use only the rankable subset.

Kendall's τ-b — what it is

A correlation coefficient for **ranked lists** that handles ties (the "-b" variant). For two rankings of the same N items: - **τ = 1.0** → identical order. - **τ = 0.0** → uncorrelated (50/50 whether any given pair is in the same order). - **τ = –1.0** → fully reversed.

Mechanically, τ-b counts concordant pairs (both rankings agree which item is higher) minus discordant pairs, normalised so ties don't artificially inflate the numerator. Used here because the per-session DP rankings are short and frequently contain ties (e.g. Ordering DPs that tie at zero magnitude), and τ-b is the standard tie-aware variant.

Headline τ for this work: under multi-knob perturbation on the user-study rankable subset, **mean τ-b = 0.586** — meaningful positive correlation but well short of full preservation, which is why the framing is "locally robust, not globally robust".

Part 4 — Why two separate datasets? (injection vs user-study)

Likely viva question: *"Why have a separate labelled injection dataset and a separate user-study dataset? Why not just label some of the user-study sessions and have one combined dataset?"*

What each dataset is for

Dataset	Question it answers	Reported as
Injection (45 sessions, library codebase, single author)	Is the detector accurate? — per-kind precision / recall	Detector-evaluation tables (precision/recall, κ)
User-study (30 sessions, order-processing codebase, 5 participants in 2 arms)	Does dashboard feedback change behaviour? — DP rate Δ and gain-stripped score Δ across the 6-session arc	Descriptive trajectories, group means

The two datasets answer **different questions**, and merging them would break both.

Important nuance about ground truth (correction to a tempting wrong answer)

`expected_kinds` in `manifest-v2.csv` is **not** "pre-declared what I planted before recording". It's a label column filled in *post-hoc* by inspecting the recorded events under the same labelling protocol any rater would apply. The `pattern` column (e.g. `ManualExtractMethod loud`) is the scenario I set out to perform — that's the only pre-declared field, and **we don't report on it**.

So the injection set's privilege is **not** privileged ground-truth provenance. Labels in both datasets would come from the same post-hoc protocol.

The four reasons that actually hold

1. **The injection set is a designed test bed.** I picked the scenarios (`pattern`) and parameters (`strength`) specifically to provoke clean instances of each of the 4 kinds in balanced quantities. The underlying behaviour in each session is curated; user-study traces are unscripted, with multiple kinds often overlapping in one session.
2. **Balanced kind coverage by design.** ~8–16 sessions per kind in injection. User-study skewed heavily Hygiene + Manual-Refactor naturally — too few Rework / Ordering examples to compute meaningful per-kind precision/recall.
3. **Participant confound (the strongest single reason).** Feedback-arm participants can't simultaneously be (a) subjects of the behavioural study and (b) ground truth for detector accuracy, because their behaviour is the dependent variable the detector is supposed to influence. Using their sessions to validate the detector would conflate cause and measurement.
4. **Two codebases = generalisation check.** Injection uses `tool/fixtures/` (library code), user-study uses `user-study-fixture/` (order-processing). If the detector worked on the codebase I designed it against but fell over on a different codebase with different people, I'd want to know — one combined dataset on one codebase = one evaluation point.

The link between them

The two datasets are not isolated: P1 and P2 labelled the injection set **before** doing their own user-study sessions. The fact that all three raters reach $\kappa \geq 0.72$ on the injection labels is what licenses everything downstream — it shows the labelling protocol is reliable enough that the detector's "ground truth" isn't just one person's opinion.

Part 5 — Critical-reflection cues per slide (verbal scripts)

External moderators reward clear claims about what the evidence does and does not establish. Several slides now carry a tiny italic grey cue line that reminds me to deliver the longer verbal qualification below. Goal: turn every quantitative result into a bounded claim that can't be over-read.

Rule of thumb for delivery: lead with the result, *then* the bounded reading, *then* what is still missing. Never the other way around — caveat-first reads as apology.

Slides 10–13 — Per-kind detail slides (the participant-quote cards)

The italic top-right quotes are the visual cues. The longer verbal versions turn each into a design trade-off:

- **Manual-Refactor:** *"P2 found this very actionable — but the detector partly measures IDE fluency, not just refactoring quality. That's useful for developer feedback, but it's not a universal measure of refactoring quality."*
- **Ordering:** *"P2's criticism is the honest read: Ordering is most useful retrospectively. It can explain a worse route after the fact, but giving the developer enough information to choose the best order prospectively remains an open interaction-design problem."*
- **Rework:** *"P1's over-penalisation comment is fair. Not every add-then-remove is waste — undo and short experimentation can be rational. The detector needs semantic filtering or tolerance for short corrective bursts."*
- **Hygiene:** *"Both participants found this the most actionable kind. But the underlying threshold is approximate: a long stretch without a test may mean risky work, or it may mean a developer is thinking. The signal is useful, not a ground-truth safety failure."*

Slide 17 — Per-kind decision matrices (precision 1.00)

Cue on slide: *"Controlled-injection: balanced fixture, scripted behaviour — not shown on real-world refactoring sessions."*

Verbal: *"Precision is 1.00 across all four kinds. The right way to read that: the detector reliably recognises the divergence patterns I defined, on the balanced fixture I designed. These are controlled-injection results — they establish internal validity, not that 1.00 precision will transfer unchanged to ordinary mixed-intent industrial refactoring sessions."*

If asked about Ordering recall specifically: *"Ordering is the weakest result: recall is limited by a deliberately conservative validator and a bounded enumeration budget. It is currently a useful diagnostic for short, reproducible windows rather than a complete ordering analyser."*

Slide 18 — Are the alternatives actually better?

Cue on slide: *"Result under J — not yet independently validated by expert ranking."*

Verbal: *"41 of 66 synthesised alternatives strictly beat the user trajectory — about 62%. Importantly, this is a relative result under the explicit objective I defined. It shows that the synthesisers can construct alternatives that improve the chosen score; it does not yet prove that developers or experts would rank those alternatives in the same order. That construct-validity step — pairwise human ranking of alternatives — is the obvious next experiment."*

Slide 21 — Score is locally robust

Cue on slide: *"Robustness ≠ calibration. Weights not validated against downstream outcomes."*

Verbal: *"Robustness is not calibration. These experiments show that the recommendation is not fragile around my chosen weights — under both single-knob and multi-knob perturbations, the top-ranked divergence point is overwhelmingly preserved. They do not prove that these weights are uniquely correct, or that the score predicts downstream outcomes such as defect rate, review cost, or time-to-merge. That would need an external dataset of longitudinal code-quality outcomes — which doesn't currently exist for refactoring trajectories."*

Slides 23–25 — User-study results

Cue: the existing *"n = 3 vs n = 2 — directional finding, not a hypothesis test"* line already carries the headline caveat. The deeper verbal version names the remaining causal ambiguity:

Verbal (after the gain-stripped chart on Slide 25): *"The no-feedback arm makes a pure task-order explanation less convincing, because both groups followed the same playbook in the same order. But it cannot distinguish genuine improvement in refactoring practice from participants learning what this dashboard rewards. So I interpret this as evidence that feedback changed measured process discipline — not yet proof of durable real-world refactoring improvement."*

This sentence is the single most defensible reading of the user-study result. If pushed on n=3 vs n=2, this is where to land.

Slide 27 — Agent extension

The slide's own framing ("Scope limit by design — adapting detectors for agent traces is future work") already does most of the work. The mature verbal interpretation is:

Verbal: *"This is not evidence that feedback fails for agents. It is evidence that a detector built around human IDE actions does not transfer automatically to a tool-using agent trace. The transcript-level plans suggest some agents responded to feedback — but the present instrumentation was insensitive to the changes they could actually make. A useful failed transfer, not a null behavioural result."*

Slide 28 — Conclusion (and closing 20 seconds)

The slide headline is now properly qualified, and the bottom italic line states what is not yet established. The closing ~20 seconds to deliver verbatim:

> *"The contribution is not a claim to have solved refactoring quality. It is a working, reproducible method for making the path through a refactoring session observable, comparable, and discussable. The next evidence needed is expert ranking validation, a larger randomised human study, and linkage to downstream engineering outcomes."*

That ending positions the work as ambitious, original, and appropriately bounded — the combination external moderators tend to trust.

Why these cues exist (meta)

The 70–84 / 85+ reflection descriptors reward clear claims about what the evidence does and does not establish. Adding a dedicated limitations slide would be both clichéd and not feasible in 18 minutes across 28 slides. Instead, each cue is a single italic grey line on the relevant result slide, paired with a longer verbal qualification here. The cue triggers the script; the script protects against over-reading.